Design and Analysis of Algorithms

Module 2: Introduction to Big-O and Correctness Proofs

This is an open educational resource.
Feel free to submit fixes, improvements, and new material here.

Last Class

We talked about an example of a simple algorithm that was slow.

Specifically: concatenating multiple strings without being smart about where we start copying.

It was a little awkward to describe *how* slow it was. We had to calculate time units and we ended up with an expression like:

$$T = \frac{130*10000*(10000+1)}{2}$$

This class

We're going to learn Big-O notation, so that we can express times like that more clearly.

You've probably heard of Big-O before, but there are other big and small letters, too: Big-O, small-o, Big-Ω*, small-ω*, and Big-θ*. We will eventually learn the whole family!

These conventions are called *Bachmann-Landau* notation after its inventors.

* Greek letter Omega, which literally means "big O" in Greek, confusingly. It's different from Big-O, mathematically though.
** This is a lower-case omega.
*** This is an upper-case theta.
**** There is no small-θ

Empirical measurements are useful

There's nothing wrong with measuring how long an algorithm takes.

In fact: you should do that. It's a good idea. Sometimes algorithms can be theoretically fast and practically slow.

For example, there are ways of multiplying matrices that are theoretically faster than the iterative way you learned, but they are only *actually* faster if the matrix is huge.

But there's a simple problem with measuring algorithms empirically...

It depends on your computer.

Other people don't have your computer.*

Mathematicians don't want to be like: "proof of running time: go get Alice's computer, then run the algorithm ten times. It will take an average of 2 minutes when n = 10000, with a standard deviation of 1.1."

Also, it's more work to deterine how an algorithm *scales* when doing purely empirical measurement. You have to take multiple measurements to determine exponents, plus additional measurements to get a feel for how much they vary.

Doing high quality empirical benchmarks is surprisingly involved/difficult (we have a couple we discuss in later lectures: check the code in the repo to see how much work we have to do to force the compiler to not throw microbenchmark code away).

* unless you're hosting an unsecure SSH or something.

Scaling

What we really care about is *modelling* how long an algorithm takes.
Specifically, how it *scales*.

We would like a nice time function that lets us plug any input value we want and get a number that lets us compare to other input values, even if the units aren't actual time.

We also don't care about constant multiplication. Your computer may be 10 times faster than your phone; that doesn't change the underlying speed of the algorithm

Scaling (2)

But we *do* care about some kinds of *variable* multiplication.

For example, consider this time function:
$T(n) = \frac{130}{2}n(n+1) = 65n(n+1)$

What matters more? The 65 at the beginning (the constant factor), or the ns (the variable factor)?

Well, let's say n is 5:
$T(n) = 65 \times 5(5+1) = 65 \times 5(6) = 65 \times 30$

Well 65 is bigger than 30, so it seems more important.

Scaling (3)

But what about when n gets big? Say, 10,000 like before.*

$$T(n) = 65 \times 10{,}000(10{,}000 + 1) = 65 \times 10^4(10^4 + 1) = 65 \times (10^8 + 10^4)$$

$10^8 + 10^4$ is a *lot* bigger than 65. It's roughly $384{,}654$ times larger in fact.

And that ratio only gets more wild as n gets bigger.

*I moved the denominator of 2 under the 130 to combine the constant factors.

Scaling (4)

It doesn't matter what constant we pick. If n gets big enough, it will matter more than the constant.

Even if the time function is something like $T(n) = 1000n$

1000 is a large constant, but if n is, say, a million, it's no longer the dominant one.

Removing constants

So instead of writing
$T(n) = 65n(n+1)$

We could delete the constant factor $T(n) = n(n+1)$

And let's distribute the n so we can see the exponents:
$T(n) = n^2 + n$

So now, we can clearly see that this is a quadratic function

Can we really do this?

It might make you uncomfortable to just remove constant factors like that.

But consider: every machine takes a different amount of time to do something.

If Alice's computer takes 10 time units to copy a byte, then strcpy would have this time:
$T_{strcpy}(n) = 10n$

But let's say Bob's computer is faster: it only takes 2 time units:
$T_{strcpy}(n) = 2n$

This information is important, but it has nothing to do with the underlying algorithm.

We want to remove machine-specific information from the time function.

Taking it a step further

If it bothered you that we removed the constant factor, this is *really* going to bother you: we're going to remove a whole variable term.

Consider again $T(n) = n^2 + n$

What matters more, n^2 or n?

Obviously n^2, but let's really see how that plays out.

Comparing $n^2 + n$ to just n

n	$n^2 + n$	n^2	Ratio: $\frac{n^2+n}{n^2}$
1	2	1	2
10	110	100	1.1
100	10,100	10,000	1.01
10^3	1,001,000	1,000,000	1.001
10^6	$10^{12} + 10^6$	10^{12}	1.000001

That n term just does not matter

It barely changes the outcome.

Again, if one algorithm took $n^2 + n$ and another took n^2, by the time you get to a large input, you can't tell the difference.

But we don't want to abuse mathematical notation and just randomly delete variables. Without a good mathematical foundation, that could let us derive erroneous statements.

Questions?

Introducing Big-O

The Big-O of $f(n)$, written $O(f(n))$ is the set of functions that are eventually bounded by $f(n)$ if n is large enough and if we multiply by a large enough positive constant C.

That means two things: we can ignore constant multiples (by choosing a big C), and that we can assume that n is really big.

For example:

- $n \in O(n)$, because $n \leq C \cdot n$ if we choose $C = 1$ (or 2, or 3, or anything ≥ 1)
- $n \in O(2n)$, because $n \leq C \cdot 2n$ if we choose $C = 1$ (or as small as 0.5)
- $n \in O(.25n)$, because $n \leq C \cdot 0.25n$ if we choose $C \geq 4$
- $n \in O(n \log n)$, because $n \leq C \cdot n \log n$ if we choose any $C > 0$, as n gets large
- $n \in O(n^2)$, because $n \leq C \cdot n^2$ if we choose any $C > 0$, as n gets large

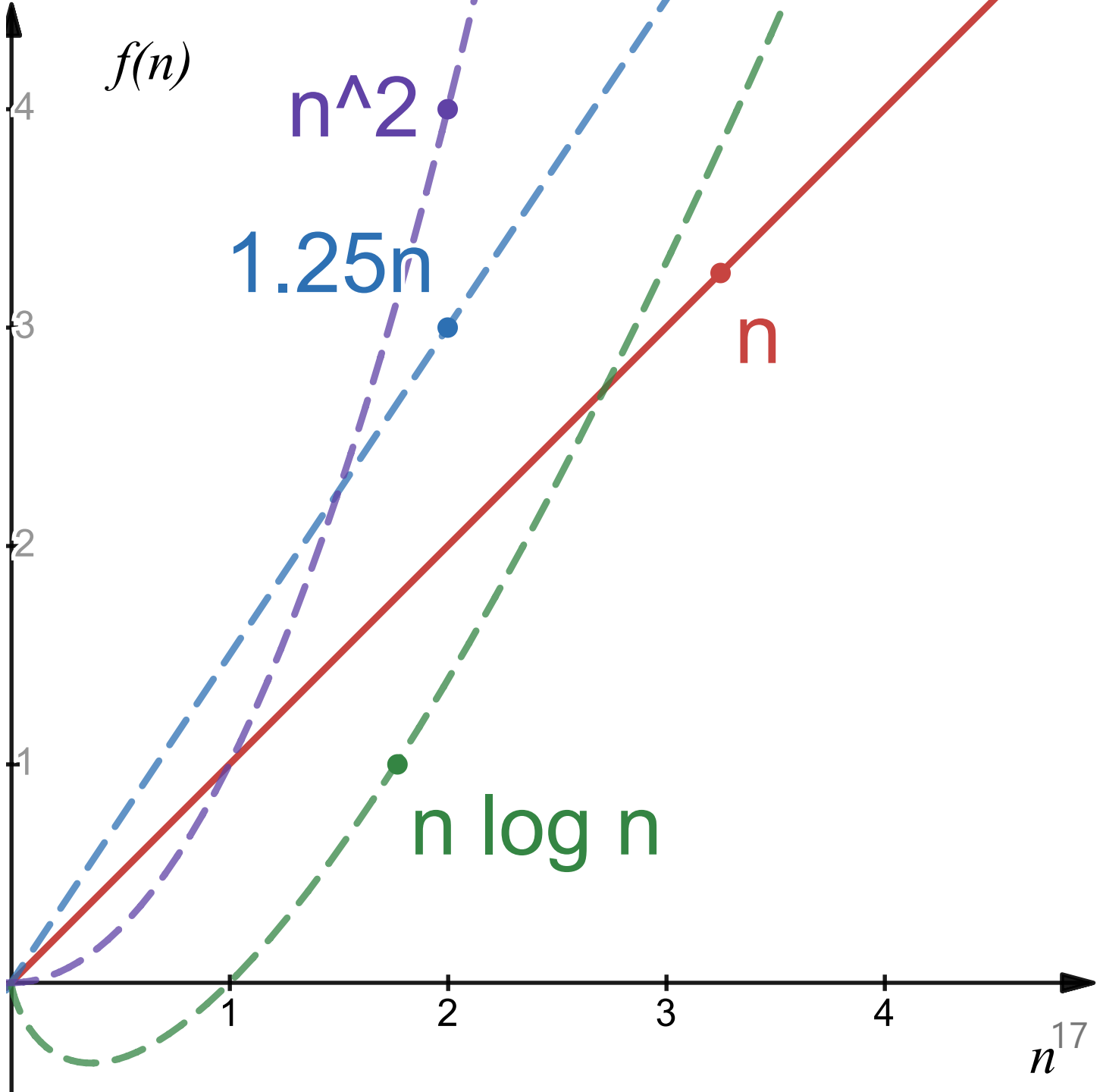
It actually doesn't matter what base the log is. The log functions with different bases all have constant ratios to each other.

A graph

Notice how, eventually all those functions get bigger than $f(n) = n$.

This would be true even of a smaller linear function, because we can multiply by any constant.

Therefore, we can say $f(n) = n \in O(n^2)$, $\in O(1.25n)$, and $\in O(n \log n)$



Even smaller functions can "bound" bigger ones

We can even say that $n \in O(0.001n)$

But how? Isn't $g(n) = 0.001n$ a lot smaller than $f(n) = n$? How could it ever be the same or larger?

Remember, $f(n) \in O(g(n))$ does not mean $g(n)$ is bigger. It means that we can *make* it be bigger if we let n be large enough *or* we multiply it by a big enough constant C!

So multiply it by 1,000! Now it's the same size and we're good.

If *some* constant can be multiplied by the function to make $g(n)$ bigger than $f(n)$ for any choice of "big enough" n, we're good. $f(n) \in O(g(n))$

Another way to think about it: $O(g(n))$ is the set of functions that can be *bounded* by $g(n)$ (with a big enough C). We're saying that f is in that set.

Questions?

Knowledge check

Give an example of a function $f(n)$, such that $n^3 \in O(f(n))$

That is, find a function $f(n)$ where $n^3 \leq C \cdot f(n)$ as n gets arbitrarily large.

Some answers

- $f(n) = n^3$
- $f(n) = \frac{n^3}{2}$
- $f(n) = n^4$
- $f(n) = n^\pi$
- $f(n) = n^3 \log n$
- $f(n) = 2^n$
- $f(n) = n!$

All of these functions can be multiplied by a constant to make them $\geq$ than n^3

Yes, even $\frac{n^3}{2}$. We can multiply it by 2!

Knowledge check (2)

Give an example of a function $f(n)$, such that $n \notin O(f(n))$

That is, find a function that is too small to have its big-O contain $g(n) = n$.

Some answers (2)

- $f(n) = log(n)$
- $f(n) = \sqrt{n}$
- $f(n) = n^{2/3}$

No matter how much we scale these functions, eventually n will be larger than them.

Example:
$n > 100\sqrt{n}$ when $n > 10000$
$n > 10000\sqrt{n}$ when $n > 1000000$
For a large enough n, for all C, $n > C\sqrt{n}$, so $n \notin O(\sqrt{n})$

Knowledge check (3)

Give an example of a function $f(n)$, such that $f(n) \in O(n^3)$

Some answers (3)

- $f(n) = n^3$
- $f(n) = n^2$
- $f(n) = n^{2/3}$
- $f(n) = n$
- $f(n) = 1$
- $f(n) = n \log n$

Yes, the functions can just be constants like 1.

The constant function $f(n) = 1 \in O(g(n))$ for any function $g(n) > 0$

Knowledge check (4)

Is $n^2 \in O(n^2 - n)$?

Some answers (4)

Actually yes. Let $C = 2$. What happens to this when n gets big?

$n^2 \leq 2(n^2 - n)$, distribute the 2
$n^2 \leq 2n^2 - 2n$, subtract n^2 from both sides
$0 \leq n^2 - 2n$
can this be made true for every n past a certain point? Add $2n$ to both sides:
$2n \leq n^2$, we only care what happens when n is big, so we can divide (assume $n > 0$)
$2 \leq n$

So $n^2 - n$ can be made to eventually bound n^2 if we multiply it by a large enough constant. Once $n \geq 2$, $2(n^2 - n)$ will always be bigger than n^2.

What Big-O *means*

Intuitively, big-O tells us the upper bound of how long a process will take.

It specifically tells us that upper bound in a way that ignores constant factors or slower growing terms.

So it's a mathematically rigorous way of removing unimportant details. "This algorithm runs in linear time (i.e., $O(n)$)" conveys a lot of information without bogging down the reader with computer specific information or meaningless time blips.

Questions?

The formal definition

Okay, let's get more rigorous. Our definition was good before, but not quite good enough for proofs.

Rigorous definition:

$$f(n) \in O(g(n)) \iff \exists C > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, f(n) \leq C \cdot g(n)$$

In English: "f is in the order of g", is equivalent to saying that there are a pair of numbers: C and n_0 which must be large enough so that $C \times g(n)$ is bigger than $f(n)$ for every choice of n that is larger than n_0.

Note: We're assuming that f and g are functions that return positive values (because they represent times). If you truly wish to allow f to return even negative numbers, you must put absolute value bars around it. We don't need to worry about this in the study of algorithms, but other fields use this notation too.

Let's meditate on that

It's a big gnarly definition. Let's think about it.

Anytime someone proves $f(n) \in O(g(n))$ constructively, they must:

- Find the scaling factor C
- Find the starting point n_0
- Prove that for every natural number starting at n_0 and getting as big as you want, that $f(n) \leq C \cdot g(n)$

We'll look at how to write one of these proofs soon, but first, let's assume someone has done the hard work for us.

Subsets example: the question

Is $O(n) \subseteq O(n^2)$?

Subsets example: the answer

Yes, it is.

But how do we prove it?

[What do you think?]

Subsets example: work (1)

What does it mean for $A \subseteq B$?

It means $\forall x \in A, x \in B$.

Another way of putting it: $\forall x \in U, x \in A \implies x \in B$

What kind of elements are we interested in? What are the elements of U? Functions. $O(f(n))$ is a set of functions $f : N \to R^+$

Therefore, we need to show:

$$\forall f : N \to R^+, f \in O(n) \implies f \in O(n^2)$$

Proving that statement

How do we prove $\forall f : N \to R^+, f \in O(n) \implies f \in O(n^2)$?

If your goal starts with $\forall$, a typical way to start your proof is by saying "suppose".

To make it easier to track, let's track the goal, and the proof so far, at the same time.

Proof so far: *empty*
Goal: $\forall f : N \to R^+, f \in O(n) \implies f \in O(n^2)$

Proving that statement (2)

Now we "suppose" to "peel off" the forall at the beginning of the goal.

Proof: "Suppose we have a function $f : N \to R^+$"

goal: ~~$\forall f : N \to R^+$~~ , $f \in O(n) \implies f \in O(n^2)$.

So, how do we prove $f \in O(n) \implies f \in O(n^2)$?

Proving implications

So how do we prove that $P \implies Q$?

We have three options:

1. Assume that P (aka, the hypothesis) is true, and show that you can prove Q (the conclusion). This is a direct proof.
2. Prove the contrapositive: $\neg Q \implies \neg P$. This is a kind of indirect proof, and it is sometimes considered less satisfying than if we could prove $P \implies Q$ directly.
3. Show that P is false. (i.e., $P \implies Q$ is vacuously true)*. Also indirect.

#3 won't work, because we have no reason to believe $f \notin O(n)$. #2 would work but indirect proofs are less "enlightening"**. Let's do #1.

* Giant footnote: see appendix A

** Another footnote: see appendix B to understand why we like direct, "constructive" proofs, especially in computer science.

Subsets example: work (2)

Proof so far: "Suppose we have a function $f : N \to R^+$"
Goal: $f \in O(n) \implies f \in O(n^2)$.

Okay, so now we "suppose" again. "Suppose" peels off an implication the same way that it peels off a "$\forall$".

Proof so far:

- Suppose we have a function $f : N \to R^+$
- Suppose that $f \in O(n)$

Goal: ~~$f \in O(n) \implies$~~ $f \in O(n^2)$

Now we have a hypothesis we can use: $f \in O(n)$. Let's use it: apply the definition of O.

Applying the definition

Recall the definition of big-O:

$$f(n) \in O(g(n)) \iff \exists C > 0, \exists n_0 \geq 0, \forall n \geq n_0, f(n) \leq C \cdot g(n)$$

And recall that our proof so far looks like this:
"Suppose we have a function $f : N \to R^+$ and suppose there is some $f \in O(n)$"

Implications are the logical version of functions*. We can "call" our definition on $f \in O(n)$ and replace it with the other side of the $\iff$

So use that big $\iff$ at the top on $f(n) \in O(g(n))$, but replace $g(n)$ with just n.

So now, we can use this fact: $\exists C > 0, \exists n_0 \geq 0, \forall n \geq n_0, f(n) \leq C \cdot g(n)$

How does that help?

* It's not a coincidence that F: P ⇒ Q looks like f : N → R. Functions are isomorphic with implications. Search for the "Curry-Howard correspondance" if this is interesting to you.

Subsets example: work (3)

Now we have a C and n_0 to work with.

Let's expand the definition in the goal, too.

Proof so far:

- Suppose we have a function $f : N \to R^+$
- Suppose $f \in O(n)$.
- Applying the definition of big-O to both the above hypothesis and goal, we derive $\exists C > 0, \exists n_0 \geq 0, \forall n \geq n_0, f(n) \leq C \cdot n$"

Goal so far:

~~$f \in O(n^2)$~~ $\exists C > 0, \exists n_0 \geq 0, \forall n \geq n_0, f(n) \leq C \cdot n^2$"

Dealing with ∃ in an assumption

We deal with ∃ and ∀ differently when they're in our "context", aka "set of assumptions" than when they are in the goal.

When an ∃ is in our set of assumptions, we can give it a name by also saying "suppose", or just break it off the proposition.

So we say "suppose" to prove a ∀ in the goal, and we say "suppose" to break off a ∃ in a hypothesis.

Let's break apart the assumption in our proof:

Subsets example: work (4)

Proof so far:

- Suppose we have a function $f : N \to R^+$
- Suppose $f \in O(n)$.
- Apply the definition of big-O to both the above hypothesis and goal, we derive $\exists C > 0, \exists n_0 \geq 0, \forall n \geq n_0, f(n) \leq C \cdot n$"
- Suppose we have a $C > 0$, $n_0 \geq 0$
- We now have $\forall n \geq n_0, f(n) \leq C \cdot n$

Goal so far:
$\exists C > 0, \exists n_0 \geq 0, \forall n \geq n_0, f(n) \leq C \cdot n^2$"

Subsets example: work (5)

Remember how saying "suppose" in our proof removed a $\forall$ from the goal?
To remove a $\exists x$ from the goal, we have to say "choose y for x".
But that requires us to actually pick a value y...what should we pick?

Remember, once we pick something, our goal will be to show
$\forall n \geq n_0', f(n) \leq C' \cdot n^2$, for some C'

And we currently have an assumption that $\forall n \geq n_0, f(n) \leq C \cdot n$

What value of C' should we pick so that $f(n) \leq C \cdot n \leq C' \cdot n^2$?

Honestly, n^2 grows much faster than n, so we can pick pretty much anything > 0. Let's pick C so the Cs cancel out when you replace the C' with it. As for n_0', we have to pick n_0, because there's a hypothesis that says $\forall n \geq n_0$ that we want to use. We could have chosen $n_0 = 0$ at the very beginning instead.

Subsets example: work (7)

Proof so far:

- Suppose we have a function $f : N \to R^+$, $f \in O(n)$.
- Apply the definition of big-O to $f \in O(n)$.
- Suppose $C > 0$, $n_0 \geq$, then $\forall n \geq n_0, f(n) \leq C \cdot n$,
  We must show $\exists C' > 0, \exists n'_0 \geq 0, \forall n' \geq n'_0, f(n') \leq C' \cdot n'$*
- Choose $C' = C$ and $n'_0 = n_0$

Goal: ~~$\exists C' > 0, \exists n'_0,$~~ $\forall n' \geq n_0, f(n') \leq C \cdot n^2$

See how we replaced the C' and the n'_0 in the goal with C and n_0? Now what to do?

* We added "primes" (the quotes after the variable names) to distinguish the variables in the goal

Subsets example: work (8)

That's right, "suppose"! $\forall n' \geq n_0, P(n)$ is shorthand for $\forall n, n \geq n_0 \implies P(n)$.
We can handle both the "$\forall$" and the $n \geq 0$ hypothesis by saying "suppose".

Proof so far:

- Suppose we have a function $f : N \to R^+$, $f \in O(n)$.
- Apply the definition of big-O to $f \in O(n)$ to obtain C and n_0.
- Suppose $C > 0$, $n_0 \geq 0$, then $\forall n, n \geq n_0 \implies f(n) \leq C \cdot n$,
  We must show $\exists C' > 0, \exists n_0' \geq 0, \forall n' \geq n_0', f(n') \leq C' \cdot n'$*
- Choose $C' = C$ and $n_0' = n_0$, and suppose we have some n', $n' \geq n_0$

Goal: ~~$\forall n' \geq n_0,$~~ $f(n') \leq C \cdot n^2$

* Here I'm repeating the goal so that the "choose" doesn't come out of nowhere.

Subsets example: what now?

Notice that we have some natural number named n' in our *context*. The context is the set of all the things we have "supposed".

And notice that we have an assumption: $\forall n, n \geq n_0 \implies f(n) \leq C \cdot n$

The "$\forall$" in the assumption means that we can apply it to any natural number we want.

Apply it to n': $n' \geq n_0 \implies f(n') \leq C \cdot n'$

Is it true that $n' \geq n_0$? Yes, we supposed it last slide. That's why we picked $n'_0 = n_0$. If it were not the case that $n \geq n_0$, we would not be allowed to apply this implication.

So now the implication gives us the hypothesis: $f(n') \leq C \cdot n'$

Now, can we *finally* show that $f(n') \leq C \cdot (n')^2$.

How?

Subsets example: by transitivity

We already have that $f(n') \leq C \cdot n'$, and it's true that $C \cdot n' \leq C \cdot (n')^2$, so, transitively,

$f(n') \leq C \cdot n' \leq C \cdot (n')^2 \implies f(n') \leq C \cdot (n')^2$, which was the goal

You can put the cool tombstone at the end: $\square$, or write *QED** if you're fancy.

* QED is an acronym which stands for "quod erat demonstratum", which means "which was to be demonstrated" in Latin.

Subsets final annotated proof

Goal: $\forall f : N \to R^+, f \in O(n) \implies f \in O(n^2)$

- Suppose we have a function $f : N \to R^+$, $f \in O(n)$.
  Goal: $f \in O(n^2)$
- Apply the definition of big-O to $f \in O(n)$ and $f \in O(n^2)$
  Suppose we have $C > 0, n_0 \geq 0, \forall n, n \geq n_0 \implies f(n) \leq C \cdot n$
  Goal: $\exists C' > 0, \exists n_0' \geq 0, \forall n', n' \geq n_0' \implies f(n') \leq C' \cdot n'$
- Choose $C' = C$ and $n_0' = n_0$, and suppose we have some n', $n' \geq n_0$
  Goal: $f(n') \leq C \cdot (n')^2$
- Apply our assumption $\forall n, n \geq n_0 \implies f(n) \leq C \cdot n$ to n'. We have that $n' \geq n_0$, so the rule applies. We derive $f(n') \leq C \cdot n'$, from which the goal, $f(n') \leq C \cdot (n')^2$ follows by transitivity of $\leq$. □

Feeling the tension

We've already seen how to prove things in discrete math, but you may not have had to do it too many times since then.

Once you see the mechanical nature of proofs, and how we say things like "suppose" in the proof to remove a hypothesis or "∀" in the goal, I hope it will feel more comfortable.

It really is like writing a computer program. In fact, when Donald Knuth was introducing computer programming, he said it was like writing a proof!

How to think about proofs

If you're struggling, I really recommend thinking about proofs in terms of "what is the proof so far" and "what is the goal so far". This is how proof assistants like Roqc work, so if you get used to this paradigm, you'll find it easy to write formal proofs if you want.

Everything you do in the proof either modifies an assumption (i.e., something we supposed) or modifies the goal (i.e., "this follows from").

Instead of trying to hold the whole proof in your brain, try to ask yourself: "okay, the proof looks like $\forall x \in Z, \ldots something$", so the next step is to say "suppose we have some $x \in Z$" and that removes the "$\forall x \in Z$ from the proof.

Just focus on one step at a time. Some steps do seem to require creativity, but you can often solve them with trial and error. Your intuition will improve with practice.

Proof tactic summary so far

- If your goal looks like $\forall x \in S, P(x)$, then say "suppose we have an $x \in S$". Now your new goal is $P(x)$ and you can use the variable you just supposed.
- You can rename a variable when you say "suppose". It's important to avoid name conflicts. If you have already "supposed" a variable named x, you cannot introduce a new variable named x.
- You can rename any $\forall$-bound variable as long as you are consistent.
- If your goal looks like $P \implies Q \ldots$, then add "suppose P". Now your goal is Q.
- If you have an **assumption** that looks like $\exists x \in S, P(x)$, you can say "suppose we have an $x \in S$ and suppose $P(x)$". Now you can use x and $P(x)$ as needed.
- If you have a **goal** that looks like $\exists x \in S, P(x)$, then you say "choose y" for some $y \in S$. Then rewrite your goal to replace x's with y's.

Proof tactic summary (2)

Proving $\neg P$ is fundamentally the same as proving $P \implies False$, so you can start by supposing P, and then deriving a contradiction.*

You can also invert the proposition and prove the inverse is true. Here is how quantified propositions get inverted:

1. $\neg \exists x, P \implies \forall x, \neg P$
2. $\neg \forall x, P \implies \exists x, \neg P$

(Note: the second one relies on non-constructive logic, so it's not as convenient with proof assistants, but we don't have to restrict ourselves in this class)

* We can think of "not P" as saying "there cannot be a proof of P". So if you had a proof of P, it would necessarily cause a contradiction.

Proof tactic summary (3)

Sometimes it's easier to work backwards than forwards. If I have a goal to show Q, and I have an assumption $P \implies Q$, I can say in my proof: "Q follows from P", and now my goal is to prove P.

I can also write: $Q \impliedby P$ which means the same thing: I used to want to prove Q, but now it's enough to prove P, because we know that if I have a proof of P, I can get Q.

Just like solving a maze, it's sometimes way easier to go backwards. Appendix E shows an example (the quadratic formula) where going backwards makes it clear what you have to do, whereas going forwards requires you to be creative.

Practice

1. Prove that $1000 \in O(1)$
2. Prove that $n^2 \notin O(n)$. I've worked this one in appendix C. I proved it by inverting quantifiers. Maybe try proving it by deriving a contradiction instead.
3. Prove that $n^2 \in O(n^2)$
   This is similar, but it's not a subset proof. Does that make a big difference?
4. Prove that $n \in O(n \lg n)$. $\lg$ is another way of writing $\log_2$
   This one is trickier. Remember that $\log_n 0$ is undefined, and $\log_n 1 = 0$
   Be careful about your choise of n_0!
5. Prove that $O(n^2 - n) = O(n^2 + n)$. I worked this one in appendix D.

Practice encouragement

Over time, you'll naturally get more comfortable with making larger leaps, and your proofs will get shorter.

Follow the long-form proof/goal method we've been using until you feel confident.

Be sure to check your work: using my worked examples when they're there, office hours/email, or AI for convenience.

Let's check the assignments to see if one is ready for us!

Questions?

Simplified notation

It gets a little tedious proving facts about big-O. It's nice to have simple rules to apply.

For example, suppose we have some algorithm that is recursive. For example, the running time of merge-sort looks like this:

$T_{ms}(0) = 1$
$T_{ms}(n) = 2T_{ms}(\frac{n}{2}) + f(n)$
where $f(n) \in O(n)$

We'll show how this is derived next module, but notice how inconvenient it is to always say "f, where f is an element of the big-O of ..."

Simplified notation (2)

It would be much nicer to just write this:
$T_{ms}(n) = 2T_{ms}(\frac{n}{2}) + O(n)$

And that's, in fact, what we do.

First, instead of saying "$f \in O(g(n))$", we say "$f = O(g(n))$"

This is an abuse of notation. It does not mean "f *is* the order of $g(n)$ (i.e., a set)". It means "f is some function in $O(g(n))$.

You've probably seen statements like $f = O(n^2)$ online. Hopefully this makes it clear that $O(n^2)$ really is a set, but we usually prefer to treat the function as a member of that set.

Simplified notation (3)

Why do we abuse the notation? Because it's nice to say $O(n) + O(n^2) = O(n^2)$ and we can only do that if $O(n)$ is treated as a member of a set instead of a set.

So here's the rule:

- When there's one big O and it's the entire right hand side:
  "$f(n) = O(g(n))$" means "$f(n) \in O(g(n))$"
- When it's a term or a factor:
  "$O(n) + O(n^2) = O(n^2)$" means
  the $O(g(n))$ terms are actually elements of that O's set.

So $O(n) + O(n^2) = O(n^2)$ from now on really means
$f + g \in O(n^2)$ where $f \in O(n)$ and $g \in O(n^2)$.

Big-O addition and multiplication simplification

There are two useful rules we will apply frequently:

1. $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
2. $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$

Remember that $O(f(n))$ and $O(g(n))$ are really *members* of a set in the above rules. Not the set itself. But $O(f(n) + g(n))$ and $O(f(n) \cdot g(n))$ *are* the sets themselves.

Practice exercise: Prove these. Choose an n_0 that is guaranteed to be larger than the n_0 for either f or g, and choose C that is guaranteed to be larger than either $f'(n) + g'(n)$ or $f'(n) \cdot g'(n)$ for any $f' \in O(f(n))$ and $g' \in O(g(n))$

Using the rules

1. Simplify $O(n) + O(n^2)$
   By the first rule, $O(n) + O(n^2) = O(n + n^2) = O(n^2)$

Notice how the higher degree term *swallows* the lower degree term with addition.

2. Simplify $O(n) \times O(\lg n)$
   By the second rule, $O(n) \times O(\lg n) = O(n \lg n)$

Notice how that's *not* the case with multiplication.

Warning #1

We *cannot* combine O over subtraction or division.

$O(n^2) - O(n^2)$ looks like 0, but it's not true for $(n^2 - n) - n^2$! Likewise for division.

Warning #2

Be *very careful* going backwards. It's sometimes reasonable to say $O(n+m) = O(n) + O(m)$, but you can get in trouble doing this.

Later we will see recursive time-functions where adding a constant factor to one expression ends up adding a linear factor to the whole function.

In that case, It's wrong to use O to arbitrarily generate terms like this: $O(n) = O(n+1) = O(n) + O(1)$

This is even more wrong: $O(n) = O(n^2) = O(n^2 + n) = O(n^2) + O(n)$

Remember that we are abusing notation here. Try to keep track of when $O(f(n))$ is a set versus a function in the set.

Constant time

One more rule: if something always takes the same amount of time, we say it is $O(1)$

What if it takes 1000 time units?

Then choose $C = 1000, n_0 = 0, 1000 \leq 1000 \cdot 1$
Therefore $1000 = O(1)$

It was a practice exercise to prove this earlier, but I wanted to re-iterate it.

Wrapping up big-O definitions and rules

Now we know that, if someone proves that an algorithm takes time $t(n) = O(g(n))$, we have a vague idea of how much time it will take at worst.

O gives us an upper bound on its runtime.

Hopefully you can see why it's useful. There are also Ω for lower bound, and Θ for both lower and upper.

Questions?

Let's have an example

Okay, enough abstract math. Let's look at a real algorithm!

Insertion sort is a simple, but surprisingly valuable sorting algorithm.

First, let's remind ourself of its code:

Insertion Sort

```
void swap(int* a, int* b);

void ins_sort(int* arr, size_t n) {
    for (size_t i = 1; i < n; i++)
        for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
            swap(arr + j - 1, arr + j);
}
```

Note: this can be slightly sped up by avoiding unecessary copies. Think about what swap does and expand it. Could you optimize the function? Answer is in the github.

** "arr + i" is equivalent to "&arr[i]". We're doing pointer math to get the ith and jth elements of arr.

Why it works

Insertion sort tracks which part of the array is sorted and which part isn't.
i points to the *first index of the unsorted part*.

Suppose we have an array like this `4, 7, 5, 1, 3, 2, 6`
We split it into a sorted part and an unsorted part: `4` | `7, 5, 1, 3, 2, 6`
There are two arrays: `[4]` and `[7, 5, 1, 3, 2, 6]`
The first array is sorted. Singleton arrays are always sorted.

Why it works (2)

Consider the second loop:

```
for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
    swap(arr + j - 1, arr + j);
```

This loop moves the unsorted element to the left until it is no longer out of order.

Starting with `[4]` and `[7, 5, 1, 3, 2, 6]`, this loop will examine `4` and `7`.
They are in-order, so it will not swap them.

Then the loop will run on `[4, 7]` and `[5, 1, 3, 2, 6]`
7 and 5 are out of order, so they will be swapped. 4 and 5 are in order, so we stop.

Then the loop will run on `[4, 5, 7]` and `[1, 3, 2, 6]`
Practice: finish the procedure. Follow the for-loop as you do it.

Why it works (3)

Every time the inner loop finishes, one more element is in the correct place, and the sorted list grows by one.

Eventually the sorted list includes the whole list.

But how do we know? Can we prove it more rigorously?

Inductive reasoning

Now it's time for the big one! We have to use induction.

We'll review it first, but...

What is induction? Can anyone tell me?

Principles of induction

Natural numbers are a useful, simple datatype that is inductive. An inductive data type has a *finite number of constructors*.

Wait, constructors? Remember programming language design: a datatype consists of a number of constructors. Natural numbers have two:

1. 0
2. $\forall n \in N, n \implies n + 1$
   (i.e., given a natural number, add 1 to it to get the next one)

Because natural numbers are inductive, there is an algorithm that gives you a way to prove propositions about them. That is, propositions of the form, $\forall n \in N, P(n)$. This algorithm is called an "inductive principle".

Weak induction

There are many inductive principles for each inductive datatype. The most basic inductive principle for natural numbers is called *weak induction**.

Weak induction is a proof algorithm. It will generate proofs of the form $\forall n \in N, P(n)$ If you give it two proofs, it will spit out a proof of $\forall n \in N, P(n)$ for any n.

Here are the proofs we must give it:

1. $P(0)$
2. $\forall n, [P(n) \implies P(n+1)]$ (notice the brackets: we fix n first. Then, $P(n)$ for whatever choice of n also must imply $P(n+1)$

These proofs each correspond to one of the constructors of natural numbers.

* yes, there is also "strong induction". We will use it when we start working with trees and divide and conquer algorithms.

The weak induction algorithm

Because this (half-functional psuedocode) is the function that weak induction requires*

```
P(n) weak_induction(P, n, P(0) base, P(forall n, (P(n) -> P(n + 1))) ind) {
    case n of
    | 0 => return base
    | n' + 1 =>                                          // n' is the number before n
        n'_proof = weak_induction(n', base, ind);    // get proof for P(n -1)
        return ind(n, n'_proof); // use proof for P(n - 1) to get P(n)
}
```

This is a proof machine. If you say, "give me the proof that P is true for 0, it spits out $P(0)$. If you say, "give me the proof of $P(1)$", it recursively gets the proof of $P(0)$ (the base case), then it applies the inductive proof to generate $P(1)$. Works for any n.

* In a proof assistant like Roqc, weak induction is *literally* a function. It has source code. You can write your own principle of induction and use that instead, as long as it is provably total. Again, there are times when other inductive principles are more convenient, so we will see some.

The proposition we want to prove

```
void ins_sort(int* arr, size_t n) {
    for (size_t i = 1; i < n; i++)
        for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
            swap(arr + j - 1, arr + j);
}
```

- **Want to show** $\forall n, arr$, after calling `ins_sort(arr, n)`, `arr[0..n)` will be sorted.
- Because this proposition starts with $\forall n$, we can use induction on natural numbers.
- Is it true for $n = 0$? Yes, the for loop gets skipped, and `arr[0..0)` is sorted.
- Now we need to show that if it sorts `arr[0..n)`, it sorts `arr[0..n+1)`

Notation: `arr[a .. b)` means the range starting at `a` and ending at `b - 1` inclusive. This is called a "half-open" interval. A closed interval will be written `arr[a..b]`, which means `b` is included in the interval.

Induction on loops

We were able to show $P(0)$ pretty quickly, but $P(n) \implies P(n+1)$ is more involved.

We want to "wrap" our loop with a property. We want that if $P(n)$ is true before entering the loop, then $P(n+1)$ will be true after.

Usually this means something like:
"if `arr[0..n)` is sorted ... one iteration happens ... now `arr[0..n+1)` is sorted

It's important that the first part *stay true before the loop, and after each iteration of the loop*. It doesn't help us if `arr[0..n)` stops being sorted, because then we're not making progress towards `arr[0..n+1)`.

This kind of property, one that is true before a loop and after each iteration, and therefore also immediately after the loop, is called a *loop invariant*.

Loop invariants

```
void ins_sort(int* arr, size_t n) {
    // base case: arr[0..1) is sorted

    // invariant: arr[0..i) is sorted
    for (size_t i = 1; i < n; i++)
        for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
            swap(arr + j - 1, arr + j);
        // now arr[0..i + 1) is sorted (we hope)
}
```

We clearly want the array to be sorted after the loop.

But there's a pesky loop inside. It needs to have this property:

`arr[0..i)` is sorted $\implies$ `arr[0..i + 1)` is sorted.

Loop invariants (2)

```
for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
    swap(arr + j - 1, arr + j);
```

This inner loop swaps elements into place, but it's a little complicated.

It has two termination conditions:

1. either it reaches the beginning of the array...
2. ...or it finds that arr[j] is in the right place.

Remember, we want: `arr[0..i)` is sorted $\implies$ `arr[0..i + 1)` is sorted.

Is there any way to relate that to the new variable, `j`?

Loop invariants (3)

```
for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
    swap(arr + j - 1, arr + j);
```

`j` partitions the array between sorted and unsorted.

Even with all this swapping: `arr[0..j)` is still sorted because we don't access $<$ j.
And, the values from `arr[j + 1..i)` were sorted before, and remain sorted after swap.
So these can be invariants.

The only issue is `arr[j-1] ++ arr[j]`, which is not sorted. But after the swap, that particular pair will be sorted.

Notation: `++` means concatinate. It's not a C operator, but the proofs would be much jankier without a nice operator like that.

Loop invariants (4)

```
// Pre: arr[0..i) is sorted
// I: arr[0..j) is sorted /\ arr[j .. i + 1) is sorted
for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
    // arr[0..j) is sorted /\ arr[j] < arr[j - 1] < arr[j + 1..i)
    // arr[0..j) < arr[j + 1..i + 1)
    swap(arr + j - 1, arr + j);
    // arr[0..j - 1) is sorted, arr[j - 1..i + 1) is sorted
    //                          ^^^^^^^^^^^^^^^^^^^^ important bit
// After termination: if arr[j] >= arr[j - 1], then
//  arr[0..j-1] sorted => arr[j..i + 1) sorted => arr[j-1] <= arr[j] =>
//  arr[0..i + 1) is sorted. If j = 0, the same applies.
```

Notice that after each swap, the right partition gets larger and the left partition gets smaller. Eventually, the left partition is empty.

Our termination proof shows that no matter which way the loop terminated, we can always show that the whole array up to and including i is sorted.

Annotated loops

```
void ins_sort(int* arr, size_t n) {
    // base case: arr[0..1) is sorted

    // I: arr[0..i) is sorted
    for (size_t i = 1; i < n; i++)
        //I: arr[0..j) is sorted, arr[j+1..i) is sorted, arr[0..j) < arr[j+1..i]
        for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)
            swap(arr + j - 1, arr + j);
            // => arr [0..j - 1) sorted, arr[j..i + 1) sorted
        // now arr[0..i + 1) is sorted
}
```

Final proof

Get ready for a big, gnarly proof by induction!

Just kidding

Goal: $\forall n \in N, a \in$ `int[]`, after `ins_sort(a, n)`, `a` is sorted.

Proof:

By induction on n

- Goal: show after `ins_sort([], 0), [] is sorted

  Proof: the loop is skipped and [] is always sorted.
- Goal: show $\forall a \in$ `int[]`,

  after `ins_sort(a, n)` is sorted $\implies$ after `ins_sort(a, n + 1)` is sorted

  (i.e., if the first `n` characters are sorted, the `n+1` th will be too)

  Proof, by loop invariant (it shows `arr[0..n)` sorted $\implies$ `arr[0..n+1)` sorted)

Do not succomb to hubris

> Beware of bugs in the above code; I have only proved it correct, not tried it.
> - Donald Knuth

"Awesome, we proved it! That means I don't need unit tests!"

Not quite:

- Our proof could be formalized incorrectly. I.e., we picked the wrong goal.
- We could have made a logic error in the proof.
- We could have made a typo in the code
- Some aspect of C semantics may be different than we expect.
- I had bugs in both my proof and code making these slides.

Proofs aid unit testing

However, it's still very useful to do this. For real. In actual code.

You don't have to decorate your loops with detailed logical arguments, but looking at the code and clearly understanding its invariant is often key to understanding it.

Once you understand the proof, you get a nice "unit test generator". You basically have a road map for every way the code could be wrong:

- If we swapped the wrong values, the inner invariant would not be true.
- If the inner loop had the wrong bounds (both starting and ending), the invariant would not be true. Test for both. Same for the outer loop.
- If by some weird fluke the increment was wrong: the invariant would not hold true.

By testing for all these things, we gain confidence that the code conforms to our model.

"How would I convince someone else?"

At this point, we're used to writing software, but not necessarily *reliable* software.

"How do I know this is right?" Right now a lot of us use vibes.

I recommend asking the question: how would I convince someone else this works?

A good invariant is a very strong argument that the approach is good.

Good unit tests that test the invariant are a strong argument that the actual code is good.

Unit testing in C

Check the code on the GitHub. Go to `code/mod2.c` .

I'm using a unit testing framework based on minunit.

It takes like, 10 lines of code to add unit testing. Just copy paste.

Feel free to copy my unit testing solution for your assignments.

Proofs are still valuable

"Seriously, why bother with proofs?"

Proofs force you to deeply understand the code.

Once you've proven it, even if the proof is wrong, when you step through the code with a debugger, it will be much easier to understand where the error is (and you can use the proof to generate asserts).

The deep understanding you get from proofs is tremendously valuable.

It's an awful feeling to go around tweaking signs and flipping inequalities to try to find the bug. Focus on understanding the inductive reasoning instead.

One more thing: most competitive and technical interview problems start from a proof.

Making good invariants

An invariant *must* have these properties:

1. Initialization: it must be true before the structure (usually a loop) is entered.
2. Maintenance: it must be true after each complete iteration.
3. Termination: It must be true when exiting the structure. This is often implied by the maintenance condition, but if you allow early breaks, you also need to consider that it's true after one of those.

But technically, `1 + 1 = 2` is an invariant that will always meet those requirements, and I won't accept it on a test. So what does an invariant need to be actually useful?

Making good invariants (2)

In general, the invariant needs to do three things:

1. Include the variables actually modified by the loop. If it doesn't describe things the loop changes, it's not really describing the behavior of the loop.
2. Progressively reach the goal. This is important for inductive reasoning. We want to know that after each iteration of the loop, we're one step closer. Ask yourself: is the property I want now true for at least one more element in the collection or range I'm iterating over?
3. Make it easy to see that the goal is met. If our algorithm is supposed to sort things, the invariant should demonstrate that the result will be sorted. If our algorithm is supposed to count things, the invariant should show that the count is correct.

Good invariant knowledge check

Write a good invariant for this loop which sums all the elements of an array and returns 0 on an empty range. We're allowing overflow to happen:

```
uint64_t sum(uint64_t* arr, size_t n) {
    uint64_t sum = 0;
    for (size_t i = 0; i < n; i++)
        sum += arr[i]
    return sum;
}
```

Good invariant knowledge check answer

```
uint64_t sum(uint64_t* arr, size_t n) {
    uint64_t acc = 0;
    // I: acc is the sum of values from [0 .. i)
    for (size_t i = 0; i < n; i++)
        acc += arr[i]
    return acc;
}
```

Here, the invariant relates the accumulator `acc` to the sum of values up to but not including `i`.

This is trivially true before the loop: [0 .. 0) is an empty range.
After each loop iteration, we have included one more value, and `i` increases.
When the loop is finished, it is still true, and it's easy to see that `i == n`. So we have also shown that `acc is the sum of values from [0 .. n)`, which is the goal.

Good invariant knowledge check (2)

Write one for this loop, which computes the totient of a positive natural number `n` (the number of smaller natural numbers `m` which are coprime, i.e., `gcd(n, m) == 1`)

We will assume that `gcd` is defined.

```
unsigned totient(unsigned n) {
    unsigned count = 0;
    for (unsigned m = 1; m < n; m++)
        if (gcd(n, m) == 1)
            count ++;
    return count;
}
```

Good invariant knowledge check 2 answers

```
unsigned totient(unsigned n) {
    unsigned count = 0;
    // I: count contains the number of totatives (co-prime smaller numbers)
    // between the range [0 .. m).
    // 0 is never a totative, so starting at 1 is valid.
    for (unsigned m = 1; m < n; m++)
        if (gcd(n, m) == 1)
            count ++;
    return count;
}
```

More invariant practice

Write simple C programs to solve these problems using loops, and then add high-quality invariants:

1. Define a function that computes the intersection between two sets. These are arrays of booleans, where `a[20]` means that the value `20` is in set `a`, and `!a[20]` means it isn't. Write the intersection to set `c`. You may assume both sets are the same size. `void set_inter(bool* a, bool* b, bool* c, size_t size);`
2. Given an array of integers, compute the longest span of `0`s in the array. That is, `{1, 0, 0, 0, 2, 0, 0}` has a span of 3 consecutive `0`s, which is its longest span. (Hint: your invariant will cover the loop, but you also need to insure that the length of your span so far is handled correctly after the loop). This problem shows why an invariant isn't always a complete proof. `uint32_t max_span(int* arr, size_t n);`

Questions?

More Proof practice

This is selection sort:

```
// finds the index of the minimum element within first n characters
int arg_min(int* arr, size_t n);
void sel_sort(int* arr, size_t n) {
    if (n == 0) return;
    for (int i = 0; i < n; i++) {
        int min_i = i + arg_min(arr + i, n - i); // add i, because ptr offset
        swap(arr + min_i, arr + i);
    }
}
```

arr[i] is swapped with the minimum value of the array to the right.

Apply the same analysis we did to insertion sort.

You'll need to implement arg_min, and prove that it works, too. If you use a loop, consider that from `[0..i)`, your variable storing the max index is correct.

Proof practice (2)

Look up bubble-sort. Isn't it pretty similar to insertion sort?

Implement it, then prove that your implementation is correct.

Questions

But how fast is it?

How do we determine the big-O of insertion sort?

First, we need to estimate its time. Let's look at the code and try to figure out how much time it takes.

The loops take a variable amount of time depending on the input. But there's one thing that always takes the same amount of time: swap.

Why?

Swap

Here's a reasonable implementation of swap.

(You may know the fancier version using xor, but it's not faster and it has an edge case):

```
void swap(int* i, int* j) {
    int t = *i;
    *i = *j;
    *j = t;
}
```

This is what it compiles into with clang, target x64, with -O2:

```
swap:   mov     eax, dword ptr [rdi]
        mov     ecx, dword ptr [rsi]
        mov     dword ptr [rdi], ecx
        mov     dword ptr [rsi], eax
        ret
```

Swap (2)

The swap is 5 assembly instructions.

Those are just moves and a return.
Specifically 1-2 micro-op moves, and a 1-3 micro-op return. Two dependencies.
This whole thing will typically take a few cycles or so.

It takes a few cycles no matter what inputs it gets. It's not like swapping big numbers is slower than swapping small ones.

Therefore we say that swap takes *constant time*.

Big assumption

There are two common ways we can bound a simple operation, like a move or add.

1. It's contant time.
2. It depends on how big it is.

Both of these assumptions make sense. If x and y are two registers, then $x + y$ takes constant time. A couple of cycles at most. And it doesn't depend on the values.

But if x and y are megabytes long Bigints, it depends. They would have to be broken down into a multi-step addition with carries. It would take an amount of time propositional to the length of the shorter int.

Ram model vs bit length model

We have to choose a model for how much time operations will take, and our choice will affect the O.

We can choose the **RAM model**, which basically states that every time you access RAM (or a register), that counts as 1 operation. Virtually every assembly instruction does this a constant number of times, so it's reasonable to say $O(1)$ ops per instruction.

Addition is $O(1)$, because it takes 2 RAM accesses (one for each operand), and $O(2) = O(1)$.

Alternatively, there's the **bit model**. This treats every number as if it were an array of bits, and each bit modified is one op. This means adding a number to itself is $O(\lg n)$, because the number n requires at least $log_2(n)$ bits.

Ram model vs bit length model

There is a time and a place for both, but in this class
we will be using the RAM model unless otherwise noted.

Put simply: we will treat each assembly instruction as $O(1)$ (aka "constant time")

This will make it easy to compute O, and make it pretty accurate to the performance on a desktop computer for reasonable values for n.

However, if n doesn't describe the length of an array, but rather a really large integer (e.g., in RSA encryption), the bit length model ends up being a better representation of how the algorithm scales. There are reasons to use either one.

Back to insertion sort

We saw that swap was 5 assembly instructions in sequence.

$$O(1) + O(1) + O(1) + O(1) + O(1) = O(5) = O(1).$$

So lets annotate how long insertion sort takes with that knowledge:

```
void ins_sort(int* arr, size_t n) {                                  // O(?)
    for (size_t i = 1; i < n; i++)                                   // O(?)
        for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)        // O(?)
            swap(arr + j - 1, arr + j);                              // O(1)
}
```

The outer loop

```
    for (size_t i = 1; i < n; i++)
```

How many times will this run?

[Class]

The outer loop (2)

It will run exactly $n - 1$ times. $n - 1 = O(n)$

Does that mean this algorithm is $O(n)$? *No!*

It means that the outer loop multiplies the O of the inner loop by a factor of $O(n)$.

So it's *at least* $O(n)$, but in fact, it's worse than that.

The inner loop

What about the inner loop?

```
for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)   // O(?)
    swap(arr + j - 1, arr + j);                         // O(1)
```

O is about upper bounds, so we need to ask: "what is the most possible number of times this loop will run?"

It would be if `arr[j]` is smaller than anything in `arr[0..j)`, so it has to swap `i` times.

Therefore, this loop will run, at most, i times. It is $O(i)$.

The whole thing

Now we can finish annotating the function:

```
void ins_sort(int* arr, size_t n) {                                  // O(?)
    for (size_t i = 1; i < n; i++)                                   // O(n)
        for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)        // O(i)
            swap(arr + j - 1, arr + j);                              // O(1)
}
```

So it runs an outer loop $O(n)$ times, and an inner loop $O(i)$ times. Does that mean it runs $O(ni)$ times?

Technically yes, but remember, i depends on n. What is the largest i could be?
$i = O(n)$

Big-O warning: variables must be independent

We don't want an extra variable unless they are *independent*. We'll talk about how to deal with multiple independent variables next module!

I will not give full credit to $O(ni)$, because it forces the reader to know what i is, which they won't know unless they read the source.

The Bachmann-Landau statements you make must be based only on inputs to the algorithm. They should not depend on computations internal to the algorithm. The reader shouldn't have to know about i to understand the runtime bound.

Did we lose something?

Can we really just eliminate i like that? Yes.

$$\sum_{i=1}^{n-1}\sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$$

Half of a quadratic function is still a quadratic function.

So we're not missing anything important. It is accurate and reasonable to say that insertion sort is $O(n^2)$. We don't need or want to say $O(\frac{n^2-n}{2})$ because that $= O(n^2)$

However, if you're comparing two $O(n^2)$ algorithms, that factor of $\frac{1}{2}$ might *empirically* matter. It's still important, and an algorithm that is 2 times faster is still 2 times faster even if the big-O is the same. That's great! But when reporting the big-O, we omit it.

Finishing up: multiply the big-Os

```
void ins_sort(int* arr, size_t n) {                                  // O(?)
    for (size_t i = 1; i < n; i++)                                   // O(n)
        for (size_t j = i; 0 < j && arr[j] < arr[j - 1]; j--)        // O(n)
            swap(arr + j - 1, arr + j);                              // O(1)
}
```

We can now say this code is $O(n) \times O(n) \times O(1) = O(n \times n \times 1) = O(n^2)$

Because the outer loop runs $O(n)$ times, and the inner loop runs $O(n)$ times, the whole thing runs $O(n^2)$ times.

So that's it, n^2 is an upper bound on the time of insertion sort.

Multiply the Big-Os?

But what about this?

```
for (int i = 0; i < n; i++) // O(n)
    constant_time_thing();

for (int i = 0; i < n; i++) // O(n)
    constant_time_thing();
```

[Class?]

Add the big-Os

```
for (int i = 0; i < n; i++) // O(n)
    constant_time_thing();

for (int i = 0; i < n; i++) // O(n)
    constant_time_thing();
```

The code runs in sequence. So this is $O(n) + O(n) = O(n + n) = O(2n) = O(n)$

Code in sequence adds the time, code nested in loops multiplies the time.

But wait: isn't n^2 bad?

Earlier we stated that insertion sort had some nice properties.

But it looks awful. Insertion sorting an array of a million elements would take a trillion time units!? Impossible.

Think about this for next time: what if the array is already almost sorted. Then what happens to insertion sort?

And it turns out: this is a very common occurence!

So even though this seems like an academic exercise: you've actually learned to analyze a very useful algorithm.

Footnote: selection sort still sucks though.

Careful! It's not always $n^{\text{n\_nested\_loops}}$!

Quick! What's the big-O of this code!

```
#define CHUNK_LEN ... //some number

int (char* str, int chunks) {
    // each chunk is CHUNK_LEN chars
    for (int i = 0; i < chunks; i++)
        for (int j = 0; j < CHUNK_LEN; j++)
            do_something_to_char(str + i * CHUNK_LEN + j); // O(1)
}
```

Careful! It's not always $n^{\text{n\_nested\_loops}}$! (2)

You might want to say $O(\text{chunks} \cdot \text{CHUNK\_LEN})$, but that would imply that `CHUNK_LEN` is a variable. It's not, it's a constant.

This code is actually just $O(\text{chunks})$.

It's also $O(n)$, where `n` is the length of the string, because that is proportional to the number of chunks. This is a *linear* algorithm! We only touch each character once.

Why do this? Lots of operations are faster if we can do them on a chunk of adjacent memory. The above was a goofy example, but chunking absolutely happens.
Don't be fooled about the big-O

Be reasonable about constant time

Consider this function

```
void linear_time_function(int n) {
    for (int i = 0; i < n; i++)
        do_constant_time_thing();
}
```

Most people would consider this to be $O(n)$.

However, the more litigious among you might say "well, n $\leq 2^{31} - 1$, so therefore, choose that times the time for `do_constant_time_thing` and it will be $O(1)$.

Consider O something that applies to abstract algorithms, not implementations. The algorithm will be parameterized by natural numbers which are infinite, not machine-specific fixed-width ints. So this is correctly stated $O(n)$.

Practice:

- Write a C function that is $O(n^3)$
- Write a C function that multiplies an $n \times n$ matrix by an n length vector. Annotate its big-O.
- Go back to your selection and bubble sort code. Annotate their big-O.

More induction practice

Prove the following propositions by induction:

- $\forall n \in N, 1 + 3 + \ldots + (2n + 1) = n^2$
  i.e., that the first n odd numbers have the sum n^2. This one has a cool visual proof.
- $\forall n \in N, 1 + 2 + \ldots + n = \frac{n(n+1)}{2}$
- $\forall (n, a, b) \in N, n \cdot (a + b) = n \cdot a + n \cdot b$
  Hint, do induction on n. Your remaining goals will be
  - $\forall (a, b) \in N, 0 \cdot (a + b) = 0 \cdot a + 0 \cdot b$ and
  - $\forall (a, b) \in N, n \cdot (a + b) = n \cdot a + n \cdot b \implies$
    $(n + 1) \cdot (a + b) = (n + 1) \cdot a + (n + 1) \cdot b$
    You can subsitute any a and b in the inductive hypothesis, but must use the given n.
    You can assume $\forall (n, m) \in N, (n + 1)(m) = n \cdot m + m$

Finally

- Go back and do the other practice exercises, especially the proof ones. You can do it!
- Read the appendices!

Questions?

Actual quiz next week

Next week we will have a quiz on this material.

This quiz counts! It's going to measure your understanding of this module.

You must bring paper and a writing implement! This is your responsibility! Set reminders on your phone!

I think when you see the practice quizzes, you're going to be a little relieved. I'm not going to make you write mega complex proofs!

Actual quiz next week (2)

Take the following practice quzzes. Time yourself!

Start studying now, and try to resolve any feelings of meta-cognitive unease. If you feel like "I don't quite get this", listen to the feeling!

Test yourself. The quiz will be proctored, pen-and-paper, and timed (15 minutes). If you aren't studying under these time and resource controls, you aren't fully studying for the quiz!

The quiz will test the first learning mastery standard.

You can also try quizzes from previous semesters. They are in this repository, in the `old_problems` folder. Note that the format of the quiz may have changed somewhat. In particular, in the first semester, I tested on big-Θ instead of big-O, which we haven't covered yet (but will next week)

Quiz: How should I study? (2)

Remember: *if you aren't studying under time controls with pen and paper,* ***you aren't fully studying!*** So actually take these like quizzes.

The first practice quiz is worked. The others aren't.

And remember to bring pen and paper for the quiz next week!

Practice Quiz 1

1. (25 points) Define an iterative function in C that returns the smallest magnitude negative int in a given list, or 0 if there are no negative numbers.
   For example `lsmall({-2, -5, 2, 5, 7, -100}, 6) == -2`. `lsmall({}, 0) == 0`
2. (50 points) Provide a useful invariant that gives us confidence the algorithm's loop is correct. It should be true before the first iteration of the loop and after each iteration of the loop. It should also relate the inputs of the function to the goal.
3. (25 points) Determine the algorithm's *tight* big-O, and prove it by putting the big-O in the margins like we did in the slides. You don't have to *prove*, but you must use the tightest big-O that fits.

(Remember: in C, to "declare" a function means to simply say that it exists along with type information like this: `void something(int x);`. To define it means to write the code inside the curly braces: `void something(int x) { ... }`.)

Practice Quiz 1 answers

You could also do it iteratively:

```
int lsmall(int* arr, size_t n) {
    int res = 0;
    // I: res is the least-magnitude negative number in arr[0..i)
    // I: (more mathematically) res = maximum (filter negatives (arr))
    for (size_t i = 0; i < n; i++)                          // O(n)
        // if we found a negative, if it's the first one or it's > res
        if (arr[i] < 0 && (res == 0 || arr[i] > res))       // O(1)
            res = arr[i];
    return res;
}
```

This algorithm is `O(n)`. The for loop runs `n` times, and the if statement inside of it is bounded by constant time.

How will I grade?

I generally grade like this:

- For extremely minor syntax errors (forgetting a semicolon, forgetting a paren where it was clearly implied, etc.) I don't take any points off.
- For minor issues like type coercions where it's wrong but it would require pretty strong knowledge of C to know about (like that signed negative overflow is undefiend behavior), I deduct 1 point.
- For small but significant logic errors (like off-by-ones), I deduct 5 points.
- For more serious logic errors but where I can still see that you understand, either 10 or 15 depending on how serious the error is.
- Point values are doubled for 50 point questions.

Practice Quiz 2

1. (25 points) Define an iterative function in C that returns the sum of every even-index element, starting with index 0. e.g., `even_sum({1, 2, 3, 4}, 4) == 4`, `even_sum({}, 0) == 0`
2. (50 points) Provide a useful invariant that gives us confidence the algorithm's loop is correct. It should be true before the first iteration of the loop and after each iteration of the loop. It should also relate the inputs of the function to the goal.
3. (25 points) Determine the algorithm's *tight* big-O, and prove it by putting the big-O in the margins like we did in the slides. You don't have to *prove*, but you must use the tightest big-O that fits.

Pratice Quiz 3

1. (25 points) Define an iterative function in C that returns the largest sum of adjacent pairs of an array. For example, `{1,2,3,1}` has adjacent pairs (1, 2); (2, 3); and (3, 1). (2, 3) has the largest sum, 5, so it would return 5.

   ```
   max_adj_sum({1,2,3,4}, 4) == 3 + 4 == 7
   ```

   ```
   max_adj_sum({1}, 1) == 0
   ```

   ```
   max_adj_sum({}, 0) == 0
   ```

2. (50 points) Provide a useful invariant that gives us confidence the algorithm's loop is correct. It should be true before the first iteration of the loop and after each iteration of the loop. It should also relate the inputs of the function to the goal.

3. (25 points) Determine the algorithm's *tight* big-O, and prove it by putting the big-O in the margins like we did in the slides. You don't have to *prove*, but you must use the tightest big-O that fits.

Practice Quiz 34

1. (25 points) Define an iterative function in C that finds the last zero-based index of the lowercase letter 'q' in an ascii string. If the letter 'q' is not present, return -1. Otherwise, return the index of the last 'q'.

   ```
   rscan_q("hello world") == -1
   ```

   ```
   rscan_q("quello quorld") == 7
   ```

   ```
   rscan_q("") == -1
   ```

2. (50 points) Provide a useful invariant that gives us confidence the algorithm's loop is correct. It should be true before the first iteration of the loop and after each iteration of the loop. It should also relate the inputs of the function to the goal.

3. (25 points) Determine the algorithm's *tight* big-O, and prove it by putting the big-O in the margins like we did in the slides. You don't have to *prove*, but you must use the tightest big-O that fits.

More practice

Do the older quizzes! I've posted last year's problems.

They're in the `old_problems` folder in the repo.

These were the actual problems (with maybe some slight wording differences) I used for the ME graded assessments.

Note: The first semester under this new outcome-based system I asked about big-Θ on quiz 1 which this semester we haven't covered yet. Replace big-Θ with tight big-O for now. The midterm and final can cover the other notation besides big-O. They can also ask for recursive functions (which we will show how to analyze next time)

They are named fairly boringly. Just a number for each one. The number is just the order of problems I made for that outcome. It doesn't mean anything specific.

Even more practice

Give this markdown file to an LLM, along with the problems from last year.

Ask it to generate a quiz like the these.

Then, pull out a pen and paper and set a timer (both important) and try to take its quiz.

Transcribe your pen-and-paper quiz and have the LLM grade you following my loose guidelines above. I recommend asking it to be strict.

This is a *great* way to use LLMs. They can be excellent learning tools.

A sample prompt

> Please generate a quiz like the practice quizzes in the attached slides and also in the attached problems from previous semesters. Don't reveal the answer. I will try to solve it, and please score me afterwards. Be fairly strict.

Note: I like linear time problems for these, but they aren't guaranteed to be linear! Try writing a quiz based around binary search for an example of a fair problem that isn't $\Theta(n)$.

Appendix A: principle of explosion

Regarding $\neg P \implies P \implies Q$.*

This is called the principle of explosion, and it often throws people for a loop. 2 + 2 = 5 → Unicorns are real, because 2 + 2 is not 5. The principle of explosion is sometimes called "ex falso quodlibet", meaning "from false, whatever you like." It comes from the fact that if a contradiction is true, e.g., P and ~P, we can create disjunctions out of thin air: P → P or Q, and then eliminate them ~P → (P or Q) -> Q, thereby proving Q from a contradiction.

We will use it sometimes, so I want to make sure it doesn't seem too weird.

* This logic is curried. In the same way that a Haskell function can be written so that it can be called $f\ p\ q$ or $f(p, q)$, logic is the same way.
$(P \implies Q \implies R) \iff P \wedge Q \implies R$

Note also: implication associates to the right:
$(P \implies Q \implies R) \iff P \implies (Q \implies R)$

Appendix A (2)

It may be upsetting, but remember, it requires us to have a proof of something that is contradictory, which should be impossible if our logic is sound. If it is still upsetting, you are in good company: some logicians and philosophers are concerned that a single untrue fact that we believe to be true can blow up our whole logical system. What if there is a true contradiction? Therefore, logicians have developed *paraconsistent* logics which do not allow the principle of explosion.

Appendix A (3)

There is a nice thing about the principle of explosion: it's like a logical equivalent to an early return.

Suppose we have some statement like $n > 5 \implies P(n)$ and we want to prove it. Our statement only applies to numbers greater than 5, so our base case for induction is easy:

by induction on n,

- when n = 0, $0 > 5 \implies P(n)$ is vacuously true
  (we early returned: sub-goal done)
- suppose $P(n)$, ... (pretend we proved $P(n+1)$)

Appendix A (4)

If we weren't allowed to explode, we'd have to modify our implication $n > 5 \implies P(n)$ to something like $P(n) \vee n \leq 5$

If we wanted to use this more complicated fact in a later proof, we'd have to consider both cases: $P(n)$ and $n \leq 5$ and complete that proof both ways.

Different paraconsistent logical systems have different ways of dealing with this. However, the fact remains that explosion is very convenient.

Compare this to programming languages where you can early return vs. languages where you have to carry out computation for every branch of an if-statement. It's kind of nice to bail as soon as something is bogus and focus on the happy path.

Appendix B: why we like direct proofs

The purpose of a proof is to convince you that a statement is true.

However, just because you are convinced doesn't necessarily mean that you deeply understand why something is true.

For example, I can make a statement like "if you have two real numbers, x and y, and $x < y$, there exists a number between them"

This statement is clearly true. But how should I prove it?

Appendix B (2)

First, let's formalize it.

$$\forall x, y \in R; x < y \implies \exists z \in R, x < z < y$$

Well, we could prove it directly:

- Suppose there are real numbers x and y, and $x < y$
  We must show $\exists z \in R, x < z < y$
- Choose $z = \frac{x+y}{2}$
  We must show $x < \frac{x+y}{2} < y$, which follows from $x < \frac{x+y}{2} \land \frac{x+y}{2} < y$
- $x < \frac{x+y}{2}$ follows from $2x < x + y$, which follows $x < y$, which was assumed.
- $\frac{x+y}{2} < y$ follows from $x + y < 2y$, which follows $x < y$, which was assumed. □

Note: We're saying "follows from" because we're modifying the goal. If I have P ⇒ Q and P in my assumptions, then I can say "Q follows from P". However, if my goal is Q and I have the assumption P ⇒ Q, then I can say "we must show Q, which follows from P" and now my new goal is P.

Appendix B (3)

That proof is nice. It doesn't just prove that there is a number between x and y, it constructs one. It says "here's the midpoint. Notice that it's bigger than x and smaller than y for any choice of x and y where $x < y$."

We can visualize it in our heads. We have a good understanding for why it's true.

Now, imagine that we proved the contrapositive.
That is, instead of: $\forall x, y \in R; P \implies Q$
We showed: $\forall x, y \in R; \neg Q \implies \neg P$

Careful: you might think "oh, we need to find some x and y where it's not true, and prove the opposite". That's how we would prove that the whole proposition is false. This proposition is true, so you won't be able to do that. The contrapositive comes after we introduce x and y, so we still have a "forall" around them.

Appendix B (4)

- Suppose there are real numbers x and y
  We must show: $\neg(\exists z \in R, x < z < y) \implies x \geq y$
- By negation of $\exists$ and de-morgans laws, this follows from
  $\forall z \in R, x \geq z \geq y \implies x \geq y$
- This follows from transitivity of $\geq \square$

It's actually shorter, but does it really explain why the original proposition is true?

Instead of "look, the midpoint is between x and y." we're saying "well, suppose there is no point between x and y, then I guess x is bigger than y". It's less direct and it doesn't give us as good of an intuition as to what's going on.

Appendix B (5)

We can guarantee that our proofs are constructive if we take classical logic but avoid using the law of excluded middle (LEM). The LEM states:

$$\forall P, P \vee \neg P$$

Where P is a proposition.

The LEM allows us to summon any proposition we want, as long as we consider its opposite. It is actually the LEM that allows us to do proofs by contrapositive. If we want to show $P \implies Q$, we can assume $Q \vee \neg Q$. If Q is true, then $P \implies Q$ is true. If $\neg Q$ is the case, then we show $\neg P$, and that makes $P \implies Q$ vacuously true.

Appendix B (6)

Fundamentally, the law of excluded middle lets us treat propositions as if they have boolean values, and we can do case matching on them.

However, when using most computer proof assistants, propositions are types, and most proofs are actually functions. That is, a proof of $P \implies Q$ is actually a function that takes a proof of P and returns a proof of Q. This is cool because it means our proofs are actually executable computer programs that operate on proofs.

A function that takes a proof of a false statement and returns a proof of a false statement is not actually as interesting: it can never be called (because you cannot obtain proof of a false statement). Therefore, there is no useful function to extract.

Appendix C: worked example

Prove that $n^2 \notin O(n)$

Here we need to prove the opposite of the normal big-O statement. If:

$$f(n) \in O(g(n)) \iff \exists C > 0, \exists n_0 \geq 0, \forall n \geq n_0, f(n) \leq C \cdot g(n)$$

Then:

$$f(n) \notin O(g(n)) \iff \forall C > 0, \forall n_0 \geq 0, \exists n \geq n_0, f(n) > C \cdot g(n)$$

Let's apply our techniques and prove this for n^2 and n.

Appendix C (2)

Goal: $\forall C > 0, \forall n_0 \geq 0, \exists n \geq n_0, n^2 > C \cdot n$

Proof:

- Suppose we have some C and n_0, $C > 0$, $n_0 \geq 0$.
  We must show: $\exists n \geq n_0, n^2 > C \cdot n$

Note: we can't just choose anything here. It has to work for *any* choice of C *and* it has to be at least as big as n_0 *and* it has to be a natural number.

What should we pick?

Note: Technically, it doesn't *have* to be as big as n0. If it's smaller, then the statement will be vacuously true. But, we don't get to pick n0, so then we'd have to the case where n0 is smaller separately. It's easier to pick something we know is bigger, because then there is only one case to prove.

Appendix C (3)

Let's choose $n = \lceil C \rceil + n_0 + 1$. This satisfies all the requirements: it's a natural number, it's certainly bigger than n_0, and it is bigger than C, too.

After choosing that value of n, our new goal is:
$(\lceil C \rceil + n_0 + 1)^2 > C \cdot (\lceil C \rceil + n_0 + 1)$

We can safely divide both sides by $(\lceil C \rceil + n_0 + 1)$, because it's clearly > 0. Our old goal follows from our new goal:

$(\lceil C \rceil + n_0 + 1) > C$

Which is clearly true. □

Note: if you don't agree that it's clearly true, try writing an intermediate value in between the two sides where it's more obvious. The goal will follow by transitivity.

Appendix C (4)

In other words, we have shown that $n^2 \notin O(n)$, because no matter what values of C and n_0 we chose, there is always a value of n that makes $n^2 > C \cdot n$.

Appendix D

Prove that $O(n^2 - n) = O(n^2 + n)$.

Goal 1: show $n^2 - n \in O(n^2 + n)$.
Proof 1: choose $C = 1, n_0 = 0$, then $n \geq 0 \implies n^2 - n \leq 1 \cdot (n^2 + n)$ by simple arithmetic.
Goal 2: show $n^2 + n \in O(n^2 - n)$.
Proof 2: choose $C = 2, n_0 = 3$. Suppose $n \geq 3$.
Then, the goal is $n^2 + n \leq 2 \cdot (n^2 - n)$, using $\impliedby$ to mean "follows from":

$$Goal \impliedby n^2 + n \leq 2n^2 - 2n \impliedby 0 \leq n^2 - 3n \impliedby 3n \leq n^2 \impliedby 3 \leq n$$

Which is an assumption. □

Appendix E: when backwards proofs are easier

Consider the quadratic formula:

$$\forall (x, a, b, c) \in R, a \neq 0,$$

$$ax + bx + c = 0 \iff x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

This is normally proved by starting with $ax + bx + c = 0$, and then applying reversable operations until we end up with $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

The main steps are to multiply $ax + bx + c = 0$ by $4a$, and then complete the square.

This requires creativity. How were you supposed to know to do this? Try working backwards instead. It's much easier.

Appendix E (2): Proof

- Suppose we have x, a, b, c, all reals, and suppose $a \neq 0$.
  We must show: $ax^2 + bx + c = 0 \iff x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
  Every operation will be invertible, and will modify the right hand side of the $\iff$.
- Multiply both sides by $2a$, $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \equiv 2ax = -b \pm \sqrt{b^2 - 4ac}$.
  We have assumed $a \neq 0$, so this is invertable.
- Add b to both sides, this is equivalent to $2ax + b = \pm sqrt(b^2 - 4ac)$
- Square both sides, this is equivalent to $(2ax + b)^2 = b^2 - 4ac$
- Expand $(2ax + b)^2$, this is equivalent to $4a^2x^2 + 4abx + b^2 = b^2 - 4ac$
- Subtract $(b^2 - 4ac)$ from both sides, we get: $4a^2x^2 + 4abx + 4ac = 0$
- Divide both sides by $4a$. We assumed $a \neq 0$. We get: $ax^2 + bx + c = 0$
- So now we have $ax^2 + bx + c = 0 \iff ax^2 + bx + c = 0$ $\square$

Appendix E (3)

The traditional proof is better for communicating *why* the quadratic formula is true.

However, we can derive the traditional proof by taking our reverse proof and then flipping it around!

So we didn't need creativity to come up with it, we just needed to start from the end.